

软件工程专业大作业

基于 MVC 架构的命令行井字棋游戏

姓名： 苏怡萌

学号： U202411889

班级： 水电 2401 班

日期： 2026 年 7 月

代码仓库：

<https://github.com/Tim-dog/Software-Engineering-Major-Assignment>

目录

1	需求描述	2
2	系统设计	2
2.1	MVC 架构划分	2
2.2	关键设计决策	2
3	UML 建模	3
3.1	用例图	3
3.2	类图	3
3.3	序列图	4
4	模块划分与类职责	6
4.1	Model 层	6
4.1.1	Board（棋盘类）	6
4.1.2	Player / HumanPlayer / AIPlayer（玩家类族）	6
4.1.3	Game（游戏核心类）	6
4.2	View 层	6
4.2.1	BoardView（视图类）	6
4.3	Controller 层	7
4.3.1	GameController（控制器类）	7
4.4	配置	7
4.5	类之间的关系	7
5	游戏运行流程	8
6	代码运行截图	8
6.1	程序启动界面（模式选择）	8
6.2	双人对战进行中	9
6.3	X 获胜	9
6.4	平局	10
6.5	人机对战	10
6.6	不合法操作提示	11
7	项目结构	11

1 需求描述

本项目要求使用 C++ 语言设计并实现一个**命令行版井字棋游戏**，采用经典的 3×3 棋盘：

- 两名玩家轮流在棋盘上落子
- 任意一方率先在横向、纵向或对角线上形成连续三个相同棋子即获胜
- 如果棋盘被填满且无人获胜，则游戏判定为平局

架构要求：采用 MVC（Model-View-Controller）架构进行设计与实现。

功能要求：

- 支持**双人对战模式**
- 支持**人机对战模式**（AI 自动选择第一个可落子的空格）
- 玩家通过**命令行输入**棋盘的行号和列号完成落子

运行环境：Windows + Visual Studio / VSCode，C++20 标准。

2 系统设计

2.1 MVC 架构划分

本项目严格遵循 MVC 三层架构：

表 1: MVC 架构层次与职责

层次	类	职责
3*Model	Board	棋盘状态管理：落子、判满、查询
	Player / HumanPlayer / AIPlayer	玩家抽象：人类输入、AI 决策（策略模式）
	Game	游戏核心逻辑：胜负判断、回合切换、状态机
View	BoardView	命令行界面渲染：棋盘绘制、状态显示
Controller	GameController	游戏主循环、协调 View 与 Model 的交互

2.2 关键设计决策

1. **玩家 X 始终由人类控制：**在人机模式下，AI 固定为玩家 O，简化设计
2. **简单 AI 策略：**从左到右、从上到下扫描第一个空格子

3. 输入校验: `HumanPlayer::play()` 循环等待合法输入, 支持格式错误、越界、占位三种异常处理
4. 游戏结束状态: `gameOver_` + `draw_` + `winner_` 三个标志完整覆盖胜负/平局/进行中三种状态

3 UML 建模

3.1 用例图

用例图如图 1 所示。玩家 1 负责开始游戏、选择模式和落子; 玩家 2 或 AI 负责落子。落子操作包含胜负判断和回合切换。

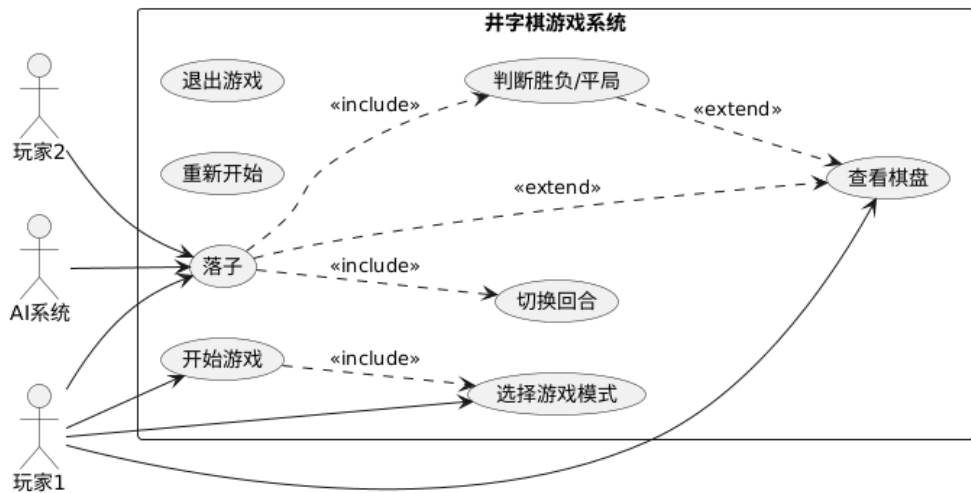


图 1: 用例图

3.2 类图

类图如图 2 所示。Model 层包含 `Board`、`Player` (及其子类 `HumanPlayer`、`AIPlayer`) 和 `Game`; View 层为 `BoardView`; Controller 层为 `GameController`。 `Player` 为抽象类, 使用多态实现人类与 AI 的差异化落子。

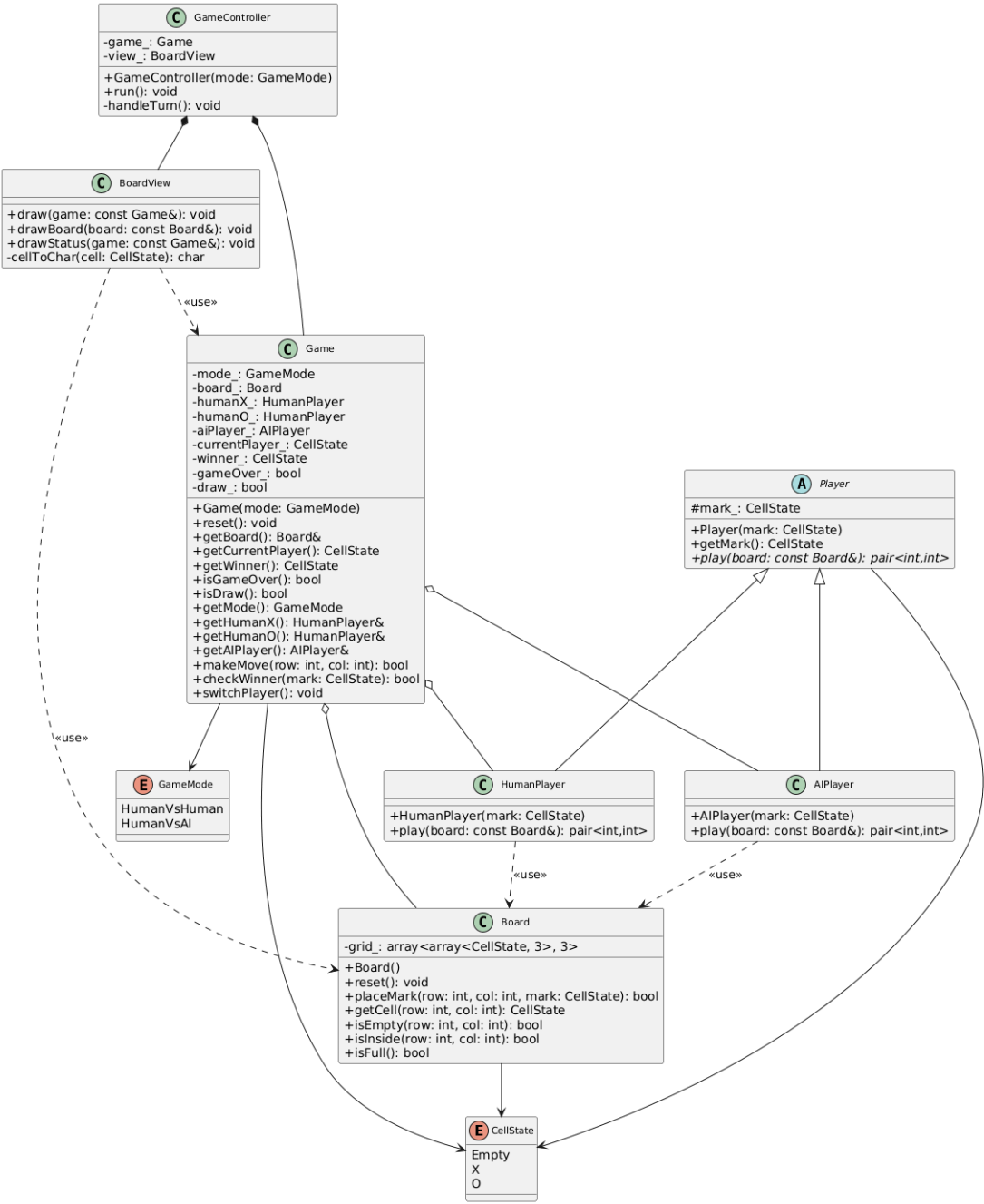
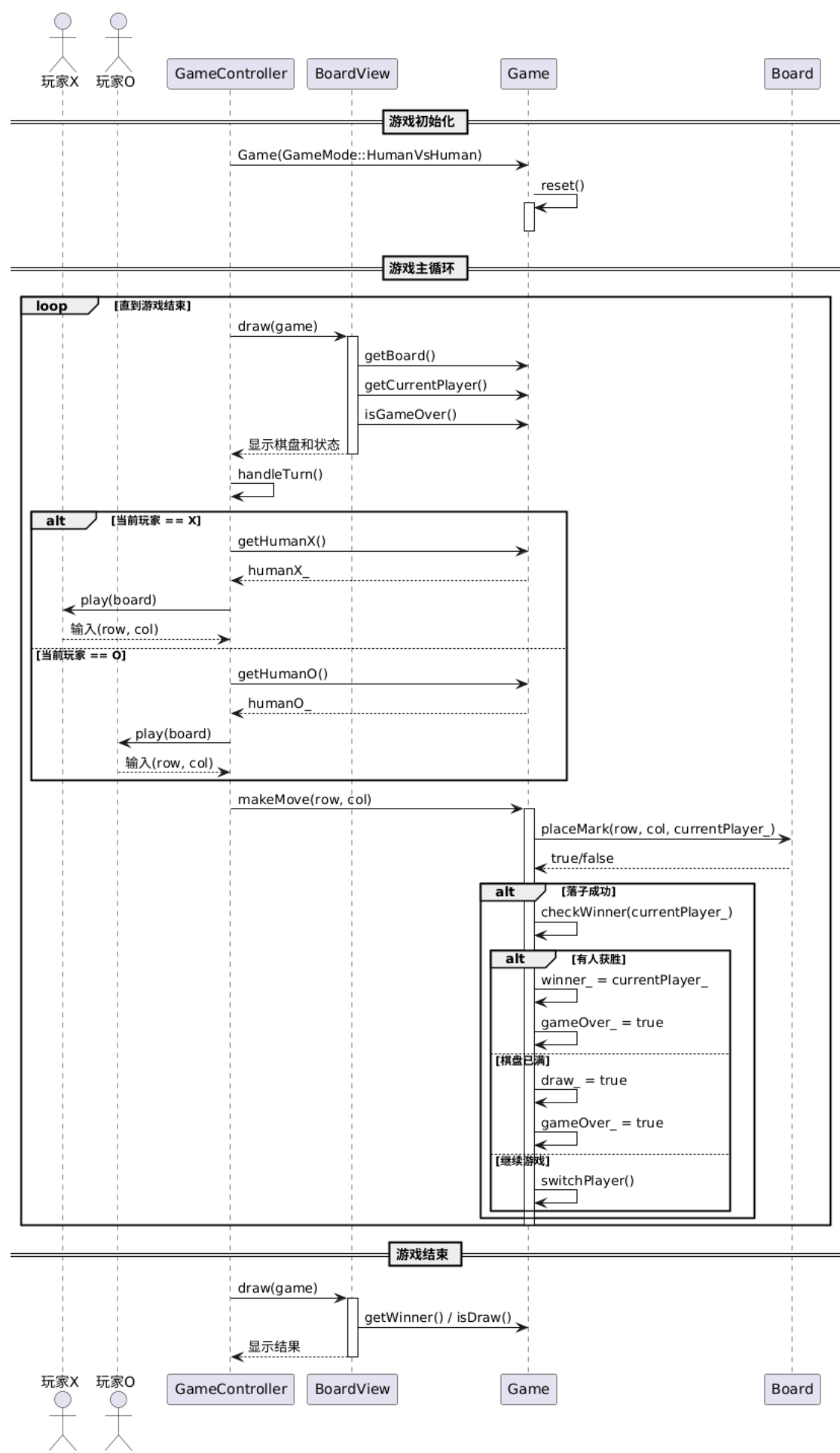


图 2: 类图（MVC 架构）

3.3 序列图

序列图如图 3 所示。展示了双人对战模式下从游戏初始化到结束的完整消息序列，包含玩家 X、玩家 O、GameController、BoardView、Game、Board 六个参与者的交互。



5
图 3: 序列图（双人对战流程）

4 模块划分与类职责

4.1 Model 层

4.1.1 Board（棋盘类）

职责：维护 3×3 棋盘状态，提供落子、查询、判满等原子操作。

关键方法：

- `placeMark(row, col, mark)` — 在指定位置落子，返回是否成功
- `getCell(row, col)` — 获取指定格子的棋子状态
- `isEmpty(row, col)` / `isInside(row, col)` — 边界和空位检查
- `isFull()` — 判断棋盘是否已满（用于平局判断）

4.1.2 Player / HumanPlayer / AIPlayer（玩家类族）

职责：抽象玩家行为，通过多态支持人类输入和 AI 决策。

设计模式：策略模式—Player 为抽象基类，`play()` 为纯虚函数。

- `HumanPlayer::play()` — 循环读取命令行输入，校验格式/范围/占用后返回坐标
- `AIPlayer::play()` — 从左到右、从上到下扫描首个空格子

4.1.3 Game（游戏核心类）

职责：管理游戏状态机，协调 Board 和 Player，判断胜负和平局。

关键方法：

- `makeMove(row, col)` — 执行一步落子的完整流程（落子 → 判胜 → 判平 → 切换回合）
- `checkWinner(mark)` — 检查 3 行、3 列、2 条对角线是否三连
- `switchPlayer()` — 在 X 和 O 之间切换当前玩家

4.2 View 层

4.2.1 BoardView（视图类）

职责：在命令行中渲染棋盘、当前玩家信息、游戏结果。

关键方法：

- `draw(game)` — 绘制完整界面（棋盘 + 状态）
- `drawBoard(board)` — 绘制 3×3 网格棋盘
- `drawStatus(game)` — 显示当前回合或游戏结果
- `cellToChar(cell)` — 将 `CellState` 枚举转换为显示字符

4.3 Controller 层

4.3.1 GameController（控制器类）

职责：驱动游戏主循环，协调 View 和 Model 的交互。

关键方法：

- `run()` — 游戏主循环：画界面 → 处理回合 → 检查结束
- `handleTurn()` — 根据当前模式和玩家选择合适的 Player 对象，调用 `play()` 获取坐标，调用 `makeMove()` 执行落子

4.4 配置

`Config.h` 定义了全局配置：

- `BOARD_SIZE = 3` — 棋盘大小常量
- `CellState` 枚举 — `Empty, X, O`
- `GameMode` 枚举 — `HumanVsHuman, HumanVsAI`

4.5 类之间的关系

<code>GameController</code>	<code>Game</code>	(组合: <code>Controller</code> 拥有 <code>Game</code>)
<code>GameController</code>	<code>BoardView</code>	(组合: <code>Controller</code> 拥有 <code>View</code>)
<code>Game</code>	<code>Board</code>	(组合: <code>Game</code> 拥有 <code>Board</code>)
<code>Game</code>	<code>HumanPlayer</code>	(聚合: <code>Game</code> 拥有 <code>HumanPlayer</code> 和 <code>AIPlayer</code>)
<code>Player < </code>	<code>HumanPlayer</code>	(继承: <code>HumanPlayer</code> 是 <code>Player</code> 的子类)
<code>Player < </code>	<code>AIPlayer</code>	(继承: <code>AIPlayer</code> 是 <code>Player</code> 的子类)
<code>BoardView ..></code>	<code>Game</code>	(依赖: <code>View</code> 读取 <code>Game</code> 状态)
<code>BoardView ..></code>	<code>Board</code>	(依赖: <code>View</code> 读取 <code>Board</code> 状态)
<code>HumanPlayer ..></code>	<code>Board</code>	(依赖: 玩家读取 <code>Board</code> 选择落子位置)
<code>AIPlayer ..></code>	<code>Board</code>	(依赖: AI 读取 <code>Board</code> 选择落子位置)

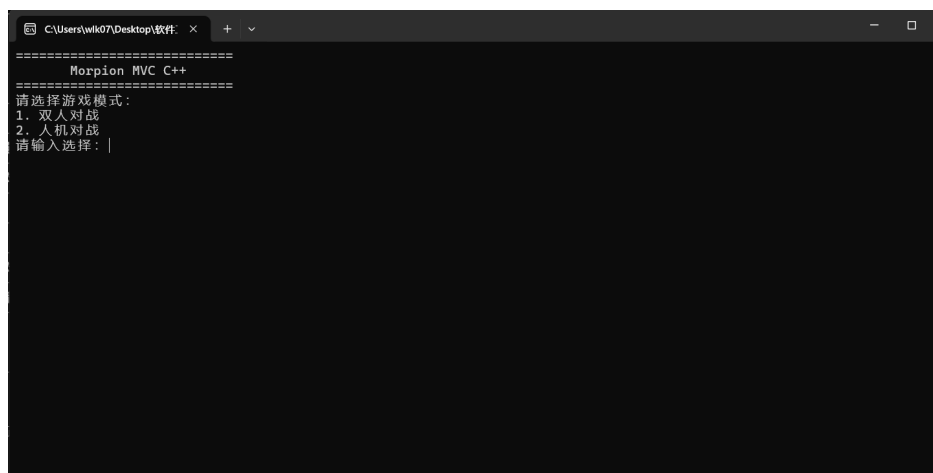
5 游戏运行流程

游戏主流程如下：

1. 程序启动，main() 设置 UTF-8 控制台编码，显示模式选择菜单
2. 用户选择 1（双人对战）或 2（人机对战）
3. GameController 初始化 Game 和 BoardView 对象
4. 进入主循环 while (!game_.isGameOver()):
 - BoardView::draw() 绘制当前棋盘和状态
 - GameController::handleTurn() 根据当前玩家和模式选择 HumanPlayer 或 AIPlayer，调用其 play() 获取落子坐标
 - Game::makeMove(row, col) 执行落子、判断胜负、判断平局、切换回合
5. 游戏结束后显示最终结果（X 胜 / O 胜 / 平局）
6. 按回车键退出程序

6 代码运行截图

6.1 程序启动界面（模式选择）



```
==== Morpion MVC C++ =====
请选择游戏模式:
1. 双人对战
2. 人机对战
请输入选择: |
```

图 4: 程序启动界面—模式选择（1= 双人对战，2= 人机对战）

6.2 双人对战进行中

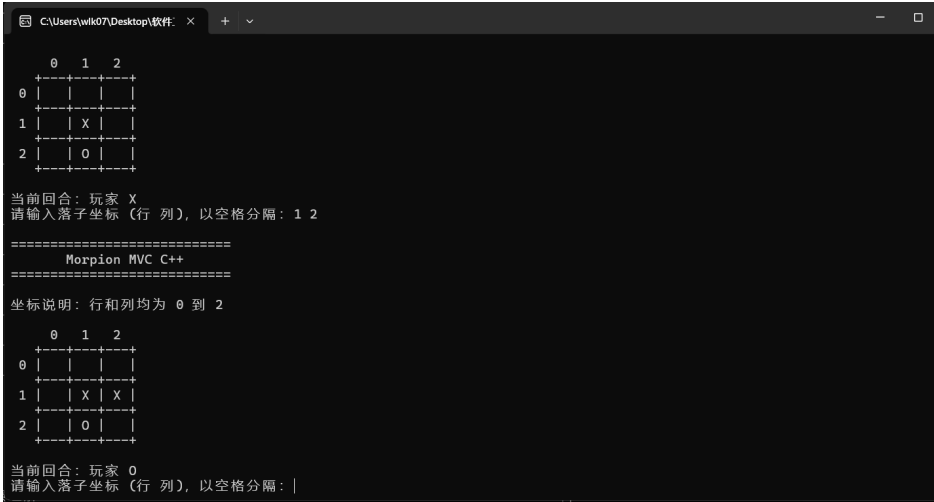


图 5: 双人对战进行中—X 与 O 交替落子

6.3 X 获胜

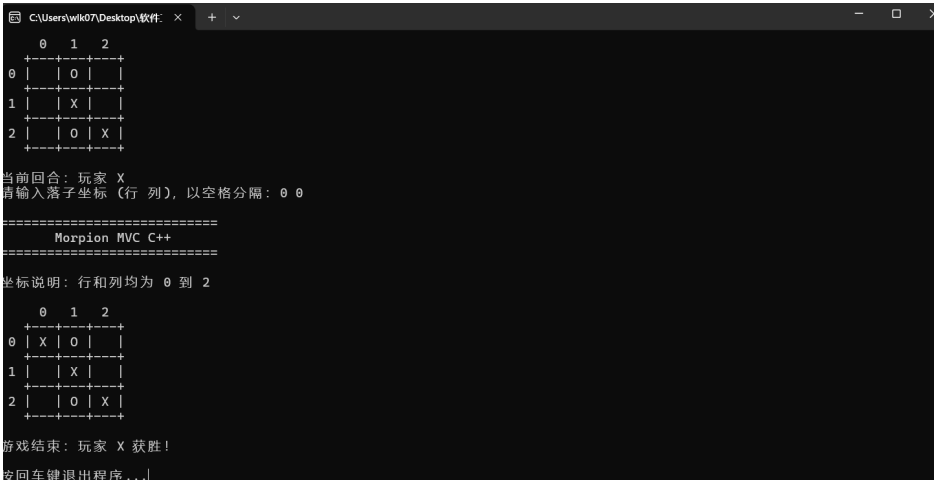


图 6: X 获胜—横向三连

6.4 平局

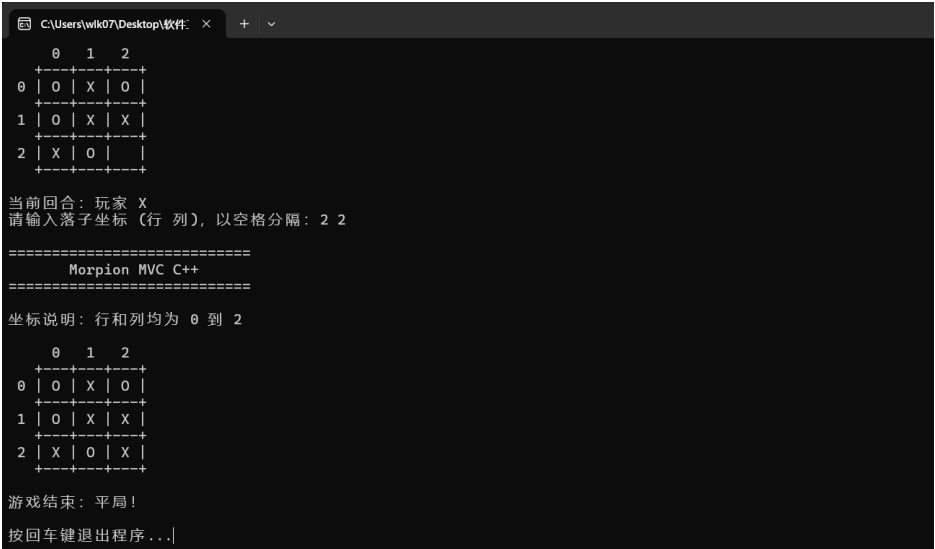


图 7: 平局—棋盘填满无人获胜

6.5 人机对战

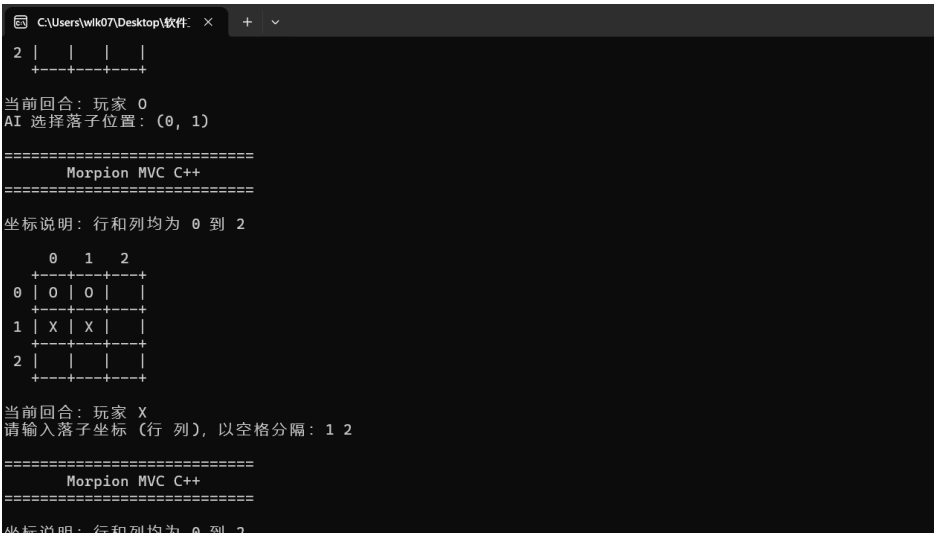


图 8: 人机对战—AI 自动选择空格落子

6.6 不合法操作提示

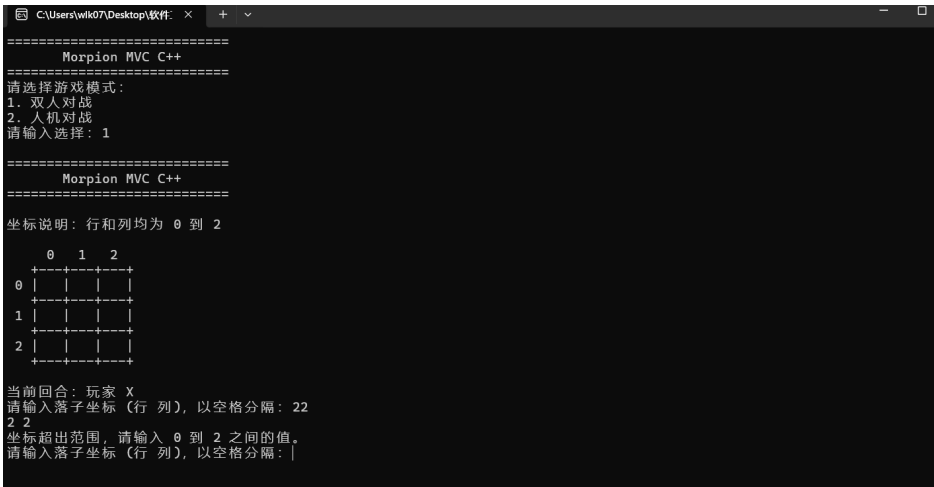


图 9: 输入校验—坐标越界或格式错误的提示信息

7 项目结构

ProjectMorpion/	
main.cpp	# 程序入口, 模式选择 + UTF-8 控制台设置
Config.h	# 全局配置 (BOARD_SIZE、CellState、GameMode 枚举)
Board.h / Board.cpp	# Model: 棋盘状态管理
Player.h / Player.cpp	# Model: 玩家抽象类 + HumanPlayer + AIPlayer
Game.h / Game.cpp	# Model: 游戏核心逻辑
BoardView.h / BoardView.cpp	# View: 命令行界面渲染
GameController.h / GameController.cpp	# Controller: 游戏主循环和回合控制
ProjectMorpion.slnx	# Visual Studio 解决方案文件
ProjectMorpion.vcxproj	# Visual Studio 项目文件
.vscode/tasks.json	# VSCode 编译任务配置 (Ctrl+Shift+B)
.gitignore	# Git 忽略规则
README.md	# 项目说明文档
report.md	# Markdown 版报告
docs/	# UML 设计文档 (PlantUML 源码)
use_case.puml	
class_diagram.puml	
sequence_diagram.puml	
figures/	# UML 渲染图片
scripts/	# 辅助脚本
软件运行情况截图/	# 游戏运行截图 (6 张)

代码仓库:<https://github.com/Tim-dog/Software-Engineering-Major-Assignment>